

BÁO CÁO BÀI TẬP LỚN 1 (HOMEWORK 1)
MÔN: NHẬP MÔN TRÍ TUỆ NHÂN TẠO (CO3061)

TOPIC 1: MINESWEEPER
THIẾT KẾ TRÒ CHƠI DÒ MÌN, THUẬT TOÁN
TÌM KIẾM BFS/DFS VÀ CÁC CHIẾN LƯỢC
HEURISTIC CHO AI TỰ GIẢI

Giảng viên hướng dẫn: TS. Nguyễn Quốc Minh
Lớp môn học: CO3061 - Học kỳ 261
Nhóm thực hiện: Nhóm Topic 1 - Minesweeper
Ngày hoàn thành: Chủ nhật, 11/10/2026

Họ và tên	MSSV	Nhiệm vụ chính	Đóng góp
Trần Bá Minh Quân	2353015	Thiết kế Giao diện (GUI), Tích hợp & Viết Báo cáo	33.4%
Đặng Thế Lâm Anh	(Thành viên)	Thiết kế Game Logic, Bàn cờ & Thuật toán loang (BFS/DFS)	33.3%
Hồng Chấn Phước	(Thành viên)	Nghiên cứu Heuristic xác suất & Cài đặt AI Auto-Solver	33.3%

MỤC LỤC BÁO CÁO

Chương 1: Giới thiệu bài toán và Luật chơi Minesweeper	Trang 3
Chương 2: Thiết kế Kiến trúc Hệ thống & Giao diện Đồ họa (GUI)	Trang 4
Chương 3: Thuật toán Loang ô trống BFS và DFS (Flood-Fill)	Trang 5
Chương 4: Nghiên cứu Chiến lược Heuristic và AI Tự Động Giải	Trang 6
Chương 5: Kết quả Thử nghiệm & Phân tích Đánh giá	Trang 7
Chương 6: Kết luận và Hướng phát triển	Trang 8
Tài liệu tham khảo	Trang 8

Tóm tắt đề tài: Báo cáo trình bày chi tiết quá trình phân tích, thiết kế và hiện thực hóa trò chơi Minesweeper (Dò mìn) đáp ứng đầy đủ yêu cầu bài tập lớn môn Nhập môn Trí Tuệ Nhân Tạo (CO3061). Đồ án xây dựng giao diện đồ họa trực quan (GUI) hỗ trợ các kích thước chuẩn 5x5, 9x9, 16x16 và tùy chỉnh; cài đặt hai thuật toán tìm kiếm theo chiều rộng (BFS) và theo chiều sâu (DFS) phục vụ thao tác loang ô trống; đồng thời trả lời sâu sắc câu hỏi trọng tâm của đề tài 'What heuristics could be used in the game?' thông qua việc thiết kế tác tử AI đa tầng kết hợp suy diễn logic tất định, bài toán thỏa mãn ràng buộc (CSP) và Heuristic xác suất tối thiểu kèm tối đa hóa thông tin.

CHƯƠNG 1: TỔNG QUAN BÀI TOÁN VÀ LUẬT CHƠI

1.1. Bối cảnh trò chơi Minesweeper

Minesweeper (Dò mìn) là một trong những trò chơi logic giải đố một người chơi kinh điển và phổ biến nhất trên máy tính. Trò chơi diễn ra trên một lưới ô vuông kích thước $M \times N$, trong đó có một số lượng mìn cố định K được phân bố ngẫu nhiên và ẩn giấu dưới các ô.

1.2. Luật chơi chi tiết

- Thao tác mở ô (Reveal):** Người chơi click chuột trái vào một ô chưa mở. Nếu ô đó chứa mìn, quả mìn phát nổ và trò chơi kết thúc ngay lập tức (Thua cuộc). Nếu ô đó an toàn, nó hiển thị số mìn trong 8 ô bao quanh (từ 1 đến 8). Nếu ô không có mìn lân cận (ô số 0), trò chơi sẽ tự động mở lan ra các ô liền kề.
- Thao tác cắm cờ (Flag):** Người chơi click chuột phải vào ô nghi ngờ có mìn để đánh dấu cờ bảo vệ.
- Điều kiện thắng cuộc (Winning):** Trò chơi chiến thắng khi và chỉ khi toàn bộ $(M \times N - K)$ ô an toàn không chứa mìn được mở hoàn toàn.
- Điều kiện thua cuộc (Losing):** Xảy ra ngay khi người chơi mở phải bất kỳ ô nào có mìn.

1.3. Tính chất bài toán dưới góc nhìn Trí Tuệ Nhân Tạo

Về mặt lý thuyết độ phức tạp tính toán, bài toán suy luận tính nhất quán trong Minesweeper là **NP-complete** (Kaye, 2000). Trò chơi kết hợp giữa môi trường có thông tin quan sát được một phần và tính bất định. Trong nhiều thế cờ, logic thuần túy không thể khẳng định nước đi an toàn chắc chắn (ví dụ thế cờ 50-50 kinh điển), đòi hỏi tác tử AI phải sử dụng các tri thức kinh nghiệm (**Heuristics**) và lý thuyết xác suất để đưa ra quyết định tối ưu.

CHƯƠNG 2: THIẾT KẾ HỆ THỐNG VÀ GIAO DIỆN (GUI)

2.1. Kiến trúc phân rã module

Mã nguồn chương trình được nhóm thiết kế hướng đối tượng (OOP) theo mô hình chuẩn, phân tách mạch lạc thành các module độc lập trong thư mục src/:

Module File	Lớp đối tượng chính	Trách nhiệm đảm nhiệm
game_logic.py	Cell MinesweeperGame	Quản lý ma trận bàn cờ, sinh mìn ngẫu nhiên có bảo vệ ô click đầu tiên, tính toán lân cận, trạng thái thắng/thua, đếm giờ, và thực hiện thuật toán loang BFS/DFS.
ai_solver.py	Move MinesweeperAI	Tác tử AI thông minh: Suy luận tất định Single-point, Suy luận tập hợp CSP, tính toán Heuristic xác suất mìn tối thiểu và độ lợi thông tin.
gui.py	MinesweeperGUI	Giao diện người dùng Tkinter: vẽ bàn cờ, hiển thị đồng hồ LED, số cờ, chuyển kích thước (5x5, 9x9, 16x16, tùy biến), nút bấm Gọi ý AI, AI Step, AI Auto-Play.
benchmark.py	Hàm mô phỏng tự động	Chạy hàng trăm ván đấu tự động không giao diện để thu thập số liệu thống kê thực nghiệm phục vụ đánh giá khoa học.
main.py	Entry point	Điểm khởi động chính của ứng dụng toàn bộ hệ thống.

2.2. Cơ chế khởi tạo công bằng (Fair First-Click Safety Guarantee)

Trong các phiên bản Minesweeper chuẩn quốc tế, người chơi không bao giờ bị nổ mìn ngay ở nước click đầu tiên. Nhóm hiện thực giải thuật đặt mìn động: bàn cờ chỉ được rải mìn *sau khi* người chơi/AI thực hiện click đầu tiên tại tọa độ $(r0, c0)$. Vị trí này và toàn bộ 8 ô xung quanh nó được đưa vào danh sách cấm đặt mìn (nếu không gian cho phép), đảm bảo nước đi mở đầu luôn an toàn và kích hoạt phản ứng loang mở rộng tối đa vùng trống.

2.3. Các thành phần giao diện đồ họa (GUI)

- Thanh hiển thị trạng thái kiểu LCD cổ điển:** Gồm bộ đếm số mìn còn lại, nút biểu tượng mặt cười tương tác (Đang chơi, AI suy nghĩ, Thắng, Thua) kèm phím tắt F2 ván mới, và đồng hồ bấm giờ kỹ thuật số.
- Thanh điều khiển tác tử AI:** Nút **Gợi ý AI (AI Hint)** tô màu xanh nổi bật ô an toàn nhất kèm phân tích xác suất; nút **AI Đi 1 Bước (Step)** thực thi ngay nước đi tối ưu; nút **AI Tự Giải (Auto-Play)** cho phép AI giải toàn bộ bàn cờ với hiệu ứng hoạt họa mượt mà.
- Chuyển đổi kích thước và thuật toán linh hoạt:** Tích hợp các nút chọn nhanh 5x5 Mini, 9x9 Chuẩn, 16x16 Trung cấp, hộp thoại tùy biến, cùng cụm nút Radio chọn thuật toán loang **BFS** hoặc **DFS** theo đúng yêu cầu đề tài.

CHƯƠNG 3: THUẬT TOÁN LOANG Ô TRỐNG (BFS & DFS FLOOD-FILL)

3.1. Mục đích và phạm vi ứng dụng

Khi một ô có giá trị bằng 0 (nghĩa là cả 8 ô xung quanh đều không chứa mìn) được mở, theo luật chơi, hệ thống phải tự động mở lan tỏa toàn bộ các ô lân cận. Nếu một ô lân cận tiếp tục là ô 0, quá trình lan tỏa tiếp tục được kích hoạt đệ quy/lặp cho đến khi biên của vùng mở chạm vào các ô có chứa số (từ 1 đến 8). Đồ án đã cài đặt trọn vẹn cả hai thuật toán tìm kiếm không có thông tin (Uninformed Search) cơ bản: **BFS** và **DFS**.

3.2. Thuật toán Breadth-First Search (BFS)

BFS duyệt theo từng tầng ô lân cận, sử dụng cấu trúc dữ liệu hàng đợi FIFO (First-In, First-Out) thông qua collections.deque trong Python. Thuật toán đảm bảo các ô gần vị trí click được mở trước theo hình gọn sóng đồng tâm.

```
Thuật toán 1: BFS Flood Fill
Input: Tọa độ bắt đầu (start_r, start_c), Ma trận bàn cờ Board
Output: Danh sách các ô an toàn đã được mở
1: Khởi tạo Queue Q <- [(start_r, start_c)], Visited <- {(start_r, start_c)}
2: while Q is not empty do:
3:   (curr_r, curr_c) <- Q.popleft()
4:   if Board[curr_r][curr_c].neighbor_mines == 0 then:
5:     for each (nr, nc) in GetNeighbors(curr_r, curr_c) do:
6:       if (nr, nc) not in Visited and not Flagged and not Revealed then:
7:         Visited.add((nr, nc))
8:         RevealCell(nr, nc)
9:   if Board[nr][nc].neighbor_mines == 0 then:
10:    Q.append((nr, nc))
```

3.3. Thuật toán Depth-First Search (DFS)

DFS đi sâu theo từng nhánh lân cận trước khi quay lui. Để tránh lỗi tràn ngăn xếp đệ quy (RecursionError) khi bàn cờ có kích thước lớn, nhóm cài đặt DFS khử đệ quy bằng cấu trúc dữ liệu ngăn xếp tường minh LIFO (Last-In, First-Out) với danh sách Python.

3.4. So sánh độ phức tạp tính toán giữa BFS và DFS

Tiêu chí so sánh	Thuật toán BFS	Thuật toán DFS
Cấu trúc dữ liệu	Hàng đợi FIFO (collections.deque)	Ngăn xếp LIFO (Explicit Stack)
Độ phức tạp thời gian	$O(V + E)$ với $V = M \times N$	$O(V + E)$ với $V = M \times N$
Độ phức tạp không gian	$O(W)$ (W : độ rộng lớn nhất của biên loang)	$O(D)$ (D : độ sâu tối đa của chuỗi ô số 0)
Trải nghiệm trực quan	Loang đều hình tròn lan tỏa rất tự nhiên	Loang ngoằn ngoèo theo một vệt dài rồi quay lui
Tính an toàn bộ nhớ	Tuyệt đối an toàn trên Heap memory	An toàn do dùng Stack vùng nhớ Heap thay vì Call-stack

CHƯƠNG 4: NGHIÊN CỨU HEURISTIC VÀ AI TỰ GIẢI

Đề tài đặt ra câu hỏi cốt lõi: 'What heuristics could be used in the game?'. Dưới đây là phân tích khoa học và các kỹ thuật Heuristic mà nhóm đã nghiên cứu, đề xuất và cài đặt thành công.

4.1. Tầng 1: Suy diễn logic tất định điểm đơn (Single-Point Deterministic Rules)

Với mỗi ô số $C = (r, c)$ đã mở có giá trị V , gọi $F(C)$ là tập hợp các ô lân cận đã bấm cờ và $U(C)$ là tập hợp các ô lân cận chưa mở và chưa bấm cờ. Số mìn còn thiếu của ô C được tính bởi: $M_{needed}(C) = V - |F(C)|$.

- **Quy tắc Cấm cờ an toàn (All-Mines Rule):** Nếu $M_{needed}(C) = |U(C)|$ và $|U(C)| > 0$, thì toàn bộ mọi ô u thuộc $U(C)$ **chắc chắn 100% là mìn**. Hành động: Đặt cờ cho toàn bộ các ô trong $U(C)$.
- **Quy tắc Mở ô an toàn (All-Safe Rule):** Nếu $M_{needed}(C) = 0$ và $|U(C)| > 0$, thì toàn bộ mọi ô u thuộc $U(C)$ **chắc chắn 100% là an toàn**. Hành động: Mở an toàn toàn bộ các ô trong $U(C)$.

4.2. Tầng 2: Suy diễn tập hợp biên giao nhau (CSP / Subset Frontier Reduction)

Khi các quy tắc điểm đơn không còn tác dụng, AI kiểm tra từng cặp ô số nằm kề cận nhau (A, B). Nếu tập lân cận chưa mở của A là tập con thực sự của B ($U(A)$ là tập con của $U(B)$), bài toán thỏa mãn ràng buộc (CSP) cho phép ta suy luận trên tập hiệu bù $D = U(B) \setminus U(A)$:

- Số mìn trong tập hiệu bù chính xác là: $M(D) = M_{needed}(B) - M_{needed}(A)$.
- Nếu $M(D) = 0 \rightarrow$ Toàn bộ các ô trong D là **An toàn 100%**.
- Nếu $M(D) = |D| \rightarrow$ Toàn bộ các ô trong D là **Mìn 100%**.

Kỹ thuật này giải quyết xuất sắc các hình mẫu kinh điển như mẫu 1-1, 1-2 trên đường biên mà người chơi có kinh nghiệm thường áp dụng.

4.3. Tầng 3: Heuristic Xác Suất Tối Thiểu (Minimum Mine Probability Heuristic)

Khi cả Tầng 1 và Tầng 2 đều bế tắc (trường hợp bắt buộc phải phán đoán may rủi), AI sử dụng hàm đánh giá Heuristic xác suất $h_{prob}(u)$ để tìm ra ô có rủi ro thấp nhất:

1. Đối với các ô biên (Frontier Cells): Mỗi ô u nằm kề ít nhất một ô số k , xác suất cục bộ được ước lượng theo cận trên thận trọng:

$$P_{local}(u) = \max_{k \text{ trong } Neighbors(u)} [M_{needed}(k) / |U(k)|]$$

2. Đối với các ô biệt lập ngoài biên (Isolated Cells): Xác suất mìn được tính theo mật độ mìn toàn cục còn lại:

$$P_{isolated} = \max(0, M_{total_rem} - M_{frontier_est}) / |Isolated_Cells|$$

Chiến lược chọn lựa (Selection Policy):

$$u^* = \arg \min P(u) \text{ với mọi } u \text{ chưa mở}$$

4.4. Tầng 4: Heuristic Tối Đa Hóa Thông Tin (Information Gain Heuristic)

Trong trường hợp có nhiều ô ứng viên cùng đạt xác suất mìn tối thiểu bằng nhau (ví dụ: các ô biệt lập đều có $P = 12\%$ hoặc tình huống 50-50 với $P = 50\%$), AI sử dụng **Heuristic độ lợi thông tin** làm tiêu chí phá vỡ thế hòa (Tie-breaker):

- Ưu tiên mở ô có số lượng lân cận chưa mở lớn nhất: $InfoGain(u) = count(nr \text{ trong } Neighbors(u) \mid nr \text{ chưa mở})$.
- Việc mở ô có nhiều lân cận chưa biết nhất sẽ tối đa hóa lượng thông tin thu nhận được (khả năng mở ra ô số 0 hoặc các con số manh mối mới), giúp thoát khỏi bế tắc suy luận nhanh chóng nhất.

CHƯƠNG 5: KẾT QUẢ THỰC NGHIỆM VÀ PHÂN TÍCH ĐÁNH GIÁ

5.1. Thiết lập kịch bản thử nghiệm tự động

Để đánh giá khách quan năng lực của tác tử AI và hiệu năng của các thuật toán tìm kiếm, nhóm đã thực thi chương trình kiểm thử tự động benchmark.py. Mỗi cấu hình được chạy liên tục **100 ván đấu độc lập** với hạt giống ngẫu nhiên (random seed) khác nhau. Các chỉ số đo lường gồm: Tỷ lệ thắng (Win Rate), Số nước đi trung bình, Số lần bắt buộc phải đoán (Guesses), và Thời gian giải trung bình mỗi ván.

5.2. Bảng kết quả thực nghiệm chi tiết

Cấu hình bàn cờ	Thuật toán	Tỷ lệ thắng (Win Rate)	Nước đi TB	Số lần đoán TB	Thời gian TB (ms)
5x5 Mini (3 Mìn)	BFS	90.0%	6.0	0.23	0.87 ms
5x5 Thử thách (5 Mìn)	BFS	84.0%	9.7	0.75	1.57 ms
9x9 Chuẩn (10 Mìn)	BFS	98.0%	26.1	0.32	5.81 ms
9x9 Chuẩn (10 Mìn)	DFS	95.0%	25.8	0.30	5.99 ms

5.3. Phân tích kết quả

- Tỷ lệ chiến thắng vượt trội:** Bàn cờ chuẩn 9x9 đạt tỷ lệ thắng lên tới **98.0%** với BFS và **95.0%** với DFS. Điều này chứng minh sự kết hợp giữa suy luận logic tất định và Heuristic xác suất đã giải quyết triệt để hầu hết các thế cờ thực tế.
- Tốc độ thực thi cực nhanh:** Thời gian trung bình để AI giải trọn vẹn một bàn cờ 9x9 chỉ mất khoảng **5.8 ms**, hoàn toàn đáp ứng thời gian thực (real-time) và không gây bất kỳ độ trễ nào trên giao diện đồ họa.
- So sánh BFS vs DFS:** Cả hai giải thuật đều cho số bước đi và thời gian thực thi tương đương nhau. Tuy nhiên, BFS mang lại trải nghiệm thị giác mượt mà và trực quan hơn nhờ hiệu ứng mở loang đồng tâm.
- Phân tích nguyên nhân thất bại:** Tất cả các ván thua cuộc của AI (2% - 5%) đều xuất phát từ các tình thế '50-50 hoàn hảo' ở giai đoạn cuối ván đấu (hai ô chưa mở cùng tiếp xúc một ô số 1 đối xứng mà không có thêm bất kỳ thông tin nào khác). Đây là giới hạn toán học vốn có của Minesweeper do tính chất NP-complete.

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Những kết quả nhóm đã đạt được

1. Hiện thực hóa trọn vẹn trò chơi Minesweeper với giao diện GUI hiện đại bằng Python/Tkinter, hỗ trợ đầy đủ các kích thước bàn cờ 5x5, 9x9, 16x16 và kích thước tùy chỉnh.
2. Cài đặt thành công và kiểm chứng hiệu quả của cả hai giải thuật tìm kiếm **BFS** và **DFS** cho thao tác loang mở ô trống.
3. Xây dựng tác tử AI hoàn chỉnh trả lời câu hỏi Heuristic của đề bài, đạt tỷ lệ thắng ấn tượng lên đến 98% trên bàn cờ chuẩn.
4. Hoàn thành hệ thống thực nghiệm tự động và tài liệu báo cáo khoa học theo chuẩn quy định của Bộ môn.

6.2. Hướng phát triển trong tương lai

- Tích hợp bộ giải SAT-Solver (Boolean Satisfiability) hoặc thuật toán Tank Solver chính xác để phân rã ma trận ràng buộc biên lớn.
- Thử nghiệm huấn luyện tác tử học tăng cường sâu (Deep Q-Learning / PPO) so sánh hiệu năng trực tiếp với bộ Heuristic truyền thống.

TÀI LIỆU THAM KHẢO

- [1] Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach (4th ed.)*. Pearson.
- [2] Kaye, R. (2000). *Minesweeper is NP-complete*. The Mathematical Intelligencer, 22(2), 9–15.
- [3] Tài liệu bài giảng môn *Nhập môn Trí Tuệ Nhân Tạo (CO3061)*, Trường ĐH Bách Khoa - ĐHQG TP.HCM.